

AgentThink Mesh — Technical Due Diligence

Document type: Internal preparation — technical DD base

Audience: Product Lead / Tech Lead, Tier 1 enterprise evaluation

Date: April 2026

Status: Draft — not for external distribution without review

1. SYSTEM ARCHITECTURE

AgentThink Mesh is structured as a modular server-side pipeline. Data enters as user-provided text or files, is processed and structured by the API layer, evaluated by the councilEngine, aggregated via consensus logic, persisted to a relational database, and returned as a structured report or UI output.

Data flow:

```
[Client Browser]
  ↓ input: text / uploaded files
[tRPC API Layer]
  ↓ typed procedure calls, server-side only
[Parsing + Structuring]
  ↓ input normalised into structured schema
[councilEngine.ts – orchestrates parallel evaluation calls]
  ↓ 10 independent LLM calls, each with a defined analytical role
[LLM API Layer]
  ↓ responses returned to councilEngine
[Consensus Logic – server-side only]
  ↓ aggregation across all perspectives
[MySQL Database – deal / session storage]
  ↓
[Report Generation + Output]
```

LLM calls occur exclusively within the councilEngine orchestration layer. The model layer is abstracted from core business logic; no LLM provider is referenced outside this boundary.

The system is modular, and components can be modified or replaced without affecting the overall workflow.

2. MODEL SELECTION & WORKFLOW

Evaluation is structured as a multi-perspective analysis across ten defined analytical roles (the “Council of 10”). Each role represents a distinct analytical lens — for example: financial risk, market positioning, regulatory exposure, operational feasibility. Roles are defined in prompt architecture, not hardcoded to any model.

- Each perspective executes independently and in parallel.
- Outputs are constrained to a structured format per role; free-form generation is not used.
- Final decision is derived via server-side consensus logic, not by any single model call.

The council architecture and consensus methodology are model-agnostic and are not dependent on any single LLM provider. The system can operate with alternative models, including locally hosted or China-compliant LLMs.

The system is designed to structure decisions, not predict outcomes.

3. DATA HANDLING & VALIDATION

All inputs are user-provided. The system does not connect to external proprietary datasets unless explicitly integrated by the operator.

- Outputs are grounded in the inputs provided. No financial data, team information, or market figures are fabricated or inferred.
- Where data is absent, the system explicitly flags the gap rather than substituting an assumption.

- No client data is used for model training.
 - Each evaluation session is isolated. No data from one client session is reused in another.
-

4. RELIABILITY & CONSISTENCY

The evaluation structure is fixed across all deals. Each perspective applies the same constrained output format regardless of input content. Aggregation occurs across multiple independent perspectives, reducing dependence on any single output.

Variance exists at the LLM inference layer and is an acknowledged property of the underlying models. The system architecture is designed to reduce this variance through structured prompting and multi-perspective aggregation.

The system is designed to reduce variability and enforce structured reasoning.

5. DATA SECURITY & COMPLIANCE

- Data is used only within the evaluation session in which it is submitted.
- No client input is reused across sessions or shared with other clients.
- No client data is used to fine-tune or train any model.
- No third-party analytics or tracking is applied to submitted deal data.

The system can be deployed in controlled environments, including private cloud or locally hosted infrastructure if required.

No SOC 2, ISO 27001, or equivalent certifications are currently in place. This is stated explicitly.

6. LIMITATIONS

- Output quality is directly dependent on input quality. Incomplete or inaccurate inputs produce incomplete or inaccurate outputs.

- The system does not guarantee correctness of any evaluation.
 - The system does not predict investment outcomes.
 - The system supports decision-making; it does not replace it.
 - LLM inference introduces non-determinism. Identical inputs may produce outputs within a controlled variance range, not identical outputs.
-

7. CORE INTELLECTUAL PROPERTY

The following components constitute the core IP of AgenThink Mesh:

- **Multi-perspective evaluation framework** — the Council of 10 architecture, role definitions, and analytical structure.
- **Prompt architecture and orchestration logic** — the structured prompt design that constrains each perspective's output format and reasoning scope.
- **Consensus and aggregation methodology** — the server-side logic that derives a final position from ten independent outputs.
- **End-to-end workflow** — the pipeline from input ingestion through evaluation to IC-format report output.

These components are independent of hosting infrastructure and are fully owned.

The codebase is version-controlled in a private GitHub repository registered to the founding entity.

8. PORTABILITY

The current deployment uses managed infrastructure provided by Manus (hosting, database, domain). This is an operational dependency, not an architectural one.

The system is built on standard, widely available components:

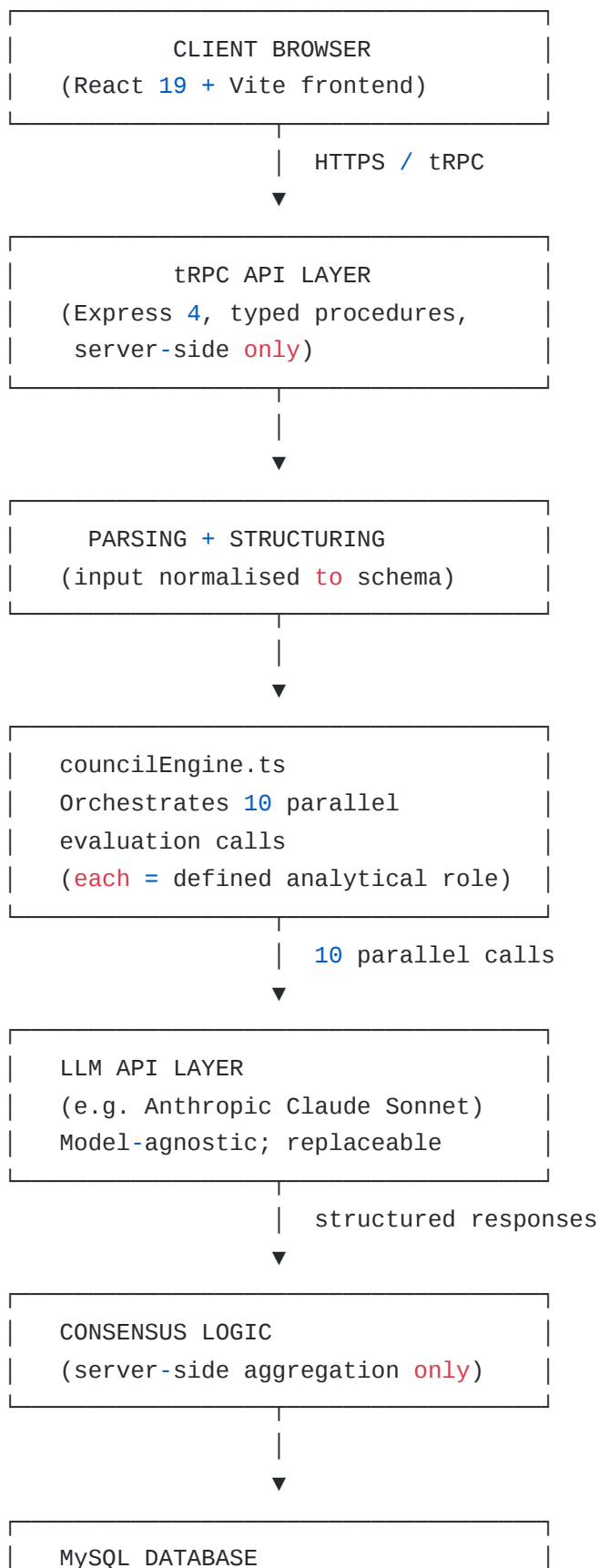
- **Backend:** Node.js (Express + tRPC)
- **Frontend:** React 19 + Vite

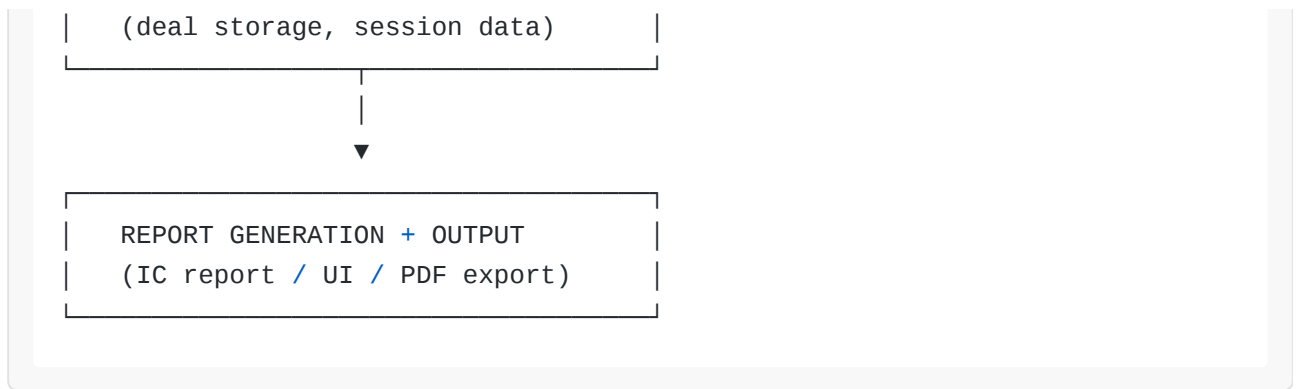
- **Database:** MySQL-compatible (currently TiDB; portable to any MySQL-compatible engine)
- **LLM integration:** abstracted API calls, replaceable with any compatible provider

No proprietary dependency prevents migration or independent deployment.

Migration to an independent cloud environment (AWS, Azure, GCP, Alibaba Cloud) requires standard DevOps work: environment provisioning, database migration, and DNS transfer. No platform-specific lock-in exists at the code level.

9. ARCHITECTURE DIAGRAM





All components above the LLM API layer run on the server. No evaluation logic executes client-side.

This document reflects the current implemented state of the system. Features not yet implemented are not described.